Table Tennis Serve Analysis — EDA

Dataset: `table_tennis_serves.csv` · 500 rows · 25 columns · 8 matches · 8 opponents

Why does a serve matter so much

Being a chopper is about constantly changing pace and being consistent. Your goal is to get the ball on the table. Sounds simple right? However, a big hurdle most choppers face is that opponents at the higher level (1800+) are quick to adapt to certain serves, and often being serves they have seen before they are able to attack with various options. So variation in the serve often times induces a slow or passive response which allows the chopping to occur and gives time for the chopper to back away from the table forcing the opponent to play a game in which the chopper is manipulating the spin.

Key Findings (read this first)

#	Finding	Implication
1	Overall win rate is **45.6%**, for a chopper facing mixed opposition	No bias fix needed
2	**Heavy spin (intensity=3) wins 55% of points** vs 37.8% for light spin	Loading up on spin, it's your biggest lever
3	Heavy backspin forces **57.6% push returns** vs 20.3% for light serves	Spin–return correlation is working correctly
4	`no_spin_disguised` to **middle_FH** wins 83% of points; same serve to elbow wins only 28%	Placement × serve type interaction is huge, especially for backhand and forehand
5	Only **31% of start_chop_rally serves** actually become chop rallies, 48% become attack rallies instead	This indicates that the came plan is needs to be adjusted as I am not controlling the rally type as much as intended setup suggests
6	Win rate when **trailing: 22%** vs leading: 65%	Momentum matters a lot
7	**Pendulum at 28.6%** of serves, you're over-indexing; no_spin_disguised and tomahawk underused (~10% each)	Diversify to reduce opponent adaptation

#	Finding	Implication
8	`direct_point` intent produces attack rally 67% of the time	Tricky serves are getting returned and attacked more than expected

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick #for % formatting
import seaborn as sns
from IPython.display import display

plt.rcParams.update({
    'figure.dpi': 120,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'font.size': 11,
})
sns.set_palette('muted')

df = pd.read_csv('table_tennis_serves.csv')
df['won'] = (df['point_outcome'] == 'won').astype(int)

print(f'Rows: {len(df)}  |  Columns: {df.shape[1]}  |  Nulls: {df.isnull().sum().sum()}')
df.head(3)
```

```
Rows: 500  |  Columns: 26  |  Nulls: 0
```

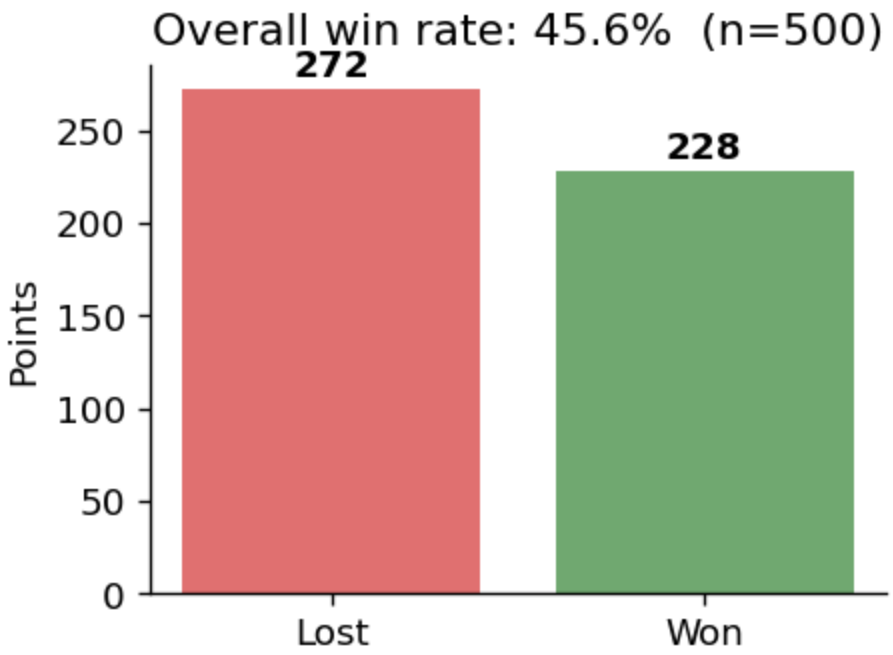
	serve_type	spin_type	spin_intensity	serve_length	placement_zone	toss_height	contact_point	match_id	ga
0	backhand	float/no-spin	1	long	middle_FH	medium	body_center	1	1
1	backhand	float/no-spin	1	long	elbow	medium	body_center	1	1
2	backhand	float/no-spin	2	long	elbow	low	hip_level_backhand	1	1

3 rows × 26 columns

1. Overall Win Rate

```
vc = df['point_outcome'].value_counts()
win_rate = df['won'].mean()

fig, ax = plt.subplots(figsize=(4, 3))
```

```
ax.bar(['Lost', 'Won'], [vc['lost'], vc['won']], color=['#e07070', '#70a870'])
ax.set_ylabel('Points')
ax.set_title(f'Overall win rate: {win_rate:.1%}  (n={len(df)})')
for bar, val in zip(ax.patches, [vc['lost'], vc['won']]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 3,
            str(val), ha='center', va='bottom', fontweight='bold')
plt.tight_layout()
plt.show()
print(f"Won: {vc['won']} ({win_rate:.1%})  |  Lost: {vc['lost']} ({1-win_rate:.1%})")
```

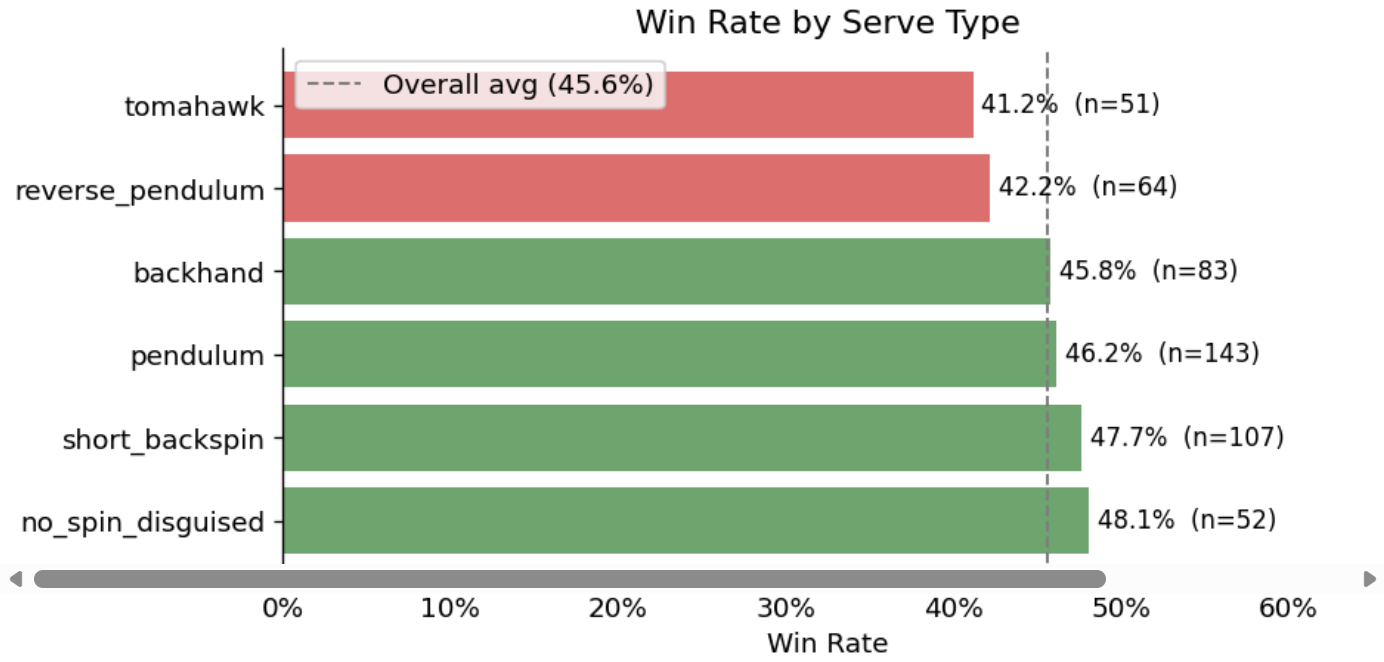



```
Won: 228 (45.6%)  |  Lost: 272 (54.4%)
```

2. Win Rate by Serve Type

```
wr_serve = (df.groupby('serve_type')['won'].agg(['mean', 'count'])
              .rename(columns={'mean': 'win_rate', 'count': 'n'})
              .sort_values('win_rate', ascending=False))

fig, ax = plt.subplots(figsize=(8, 4))
colors = ['#70a870' if v > 0.456 else '#e07070' for v in wr_serve['win_rate']]
bars = ax.barh(wr_serve.index, wr_serve['win_rate'], color=colors)
ax.axvline(df['won'].mean(), color='gray', linestyle='--', linewidth=1.2, label=f'Overall avg ({d
ax.set_xlabel('Win Rate')
ax.xaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_title('Win Rate by Serve Type')
ax.legend()
for bar, (_, row) in zip(bars, wr_serve.iterrows()):
    ax.text(row['win_rate'] + 0.005, bar.get_y() + bar.get_height()/2,
            f"{row['win_rate']:.1%}  (n={int(row['n'])})", va='center', fontsize=10)
ax.set_xlim(0, 0.65)
```

```
plt.tight_layout()
plt.show()

print('\nNote: no_spin_disguised and short_backspin lead, high-spin and deceptive serves reward m
print('reverse_pendulum and tomahawk trail, these produce more attackable returns in this dataset
```



Note: no_spin_disguised and short_backspin lead, high-spin and deceptive serves reward most.
reverse_pendulum and tomahawk trail, these produce more attackable returns in this dataset.

```
from scipy.stats import chi2_contingency

chi2_results = []
for serve_type, group in df.groupby('serve_type'):
    won_count = group['won'].sum()
    lost_count = len(group) - won_count
    n = len(group)
    win_rate_val = won_count / n
    # 2x2 contingency table: [[won, lost], [expected_won, expected_lost]]
    # Compare observed wins/losses vs expected at overall win rate
    overall_won = df['won'].sum()
    overall_lost = len(df) - overall_won
    contingency = [
        [won_count, lost_count],
        [overall_won, overall_lost]
    ]
    chi2_stat, p_value, dof, expected = chi2_contingency(contingency)
    chi2_results.append({
        'serve_type': serve_type,
        'win_rate': round(win_rate_val, 4),
```

```
        'n': n,
        'p_value': round(p_value, 4),
        'significant': p_value < 0.05
    })

chi2_df = (pd.DataFrame(chi2_results)
             .sort_values('win_rate', ascending=False)
             .reset_index(drop=True))

print('Chi-square significance test: win rate by serve type vs overall baseline')
print('=' * 70)
display(chi2_df)
print()
print('p-values below 0.05 indicate the serve type\'s win rate is significantly different from cha
```

```
Chi-square significance test: win rate by serve type vs overall baseline
======================================================================
```

	serve_type	win_rate	n	p_value	significant
0	no_spin_disguised	0.4808	52	0.8454	False
1	short_backspin	0.4766	107	0.7780	False
2	pendulum	0.4615	143	0.9824	False
3	backhand	0.4578	83	1.0000	False
4	reverse_pendulum	0.4219	64	0.7017	False
5	tomahawk	0.4118	51	0.6477	False

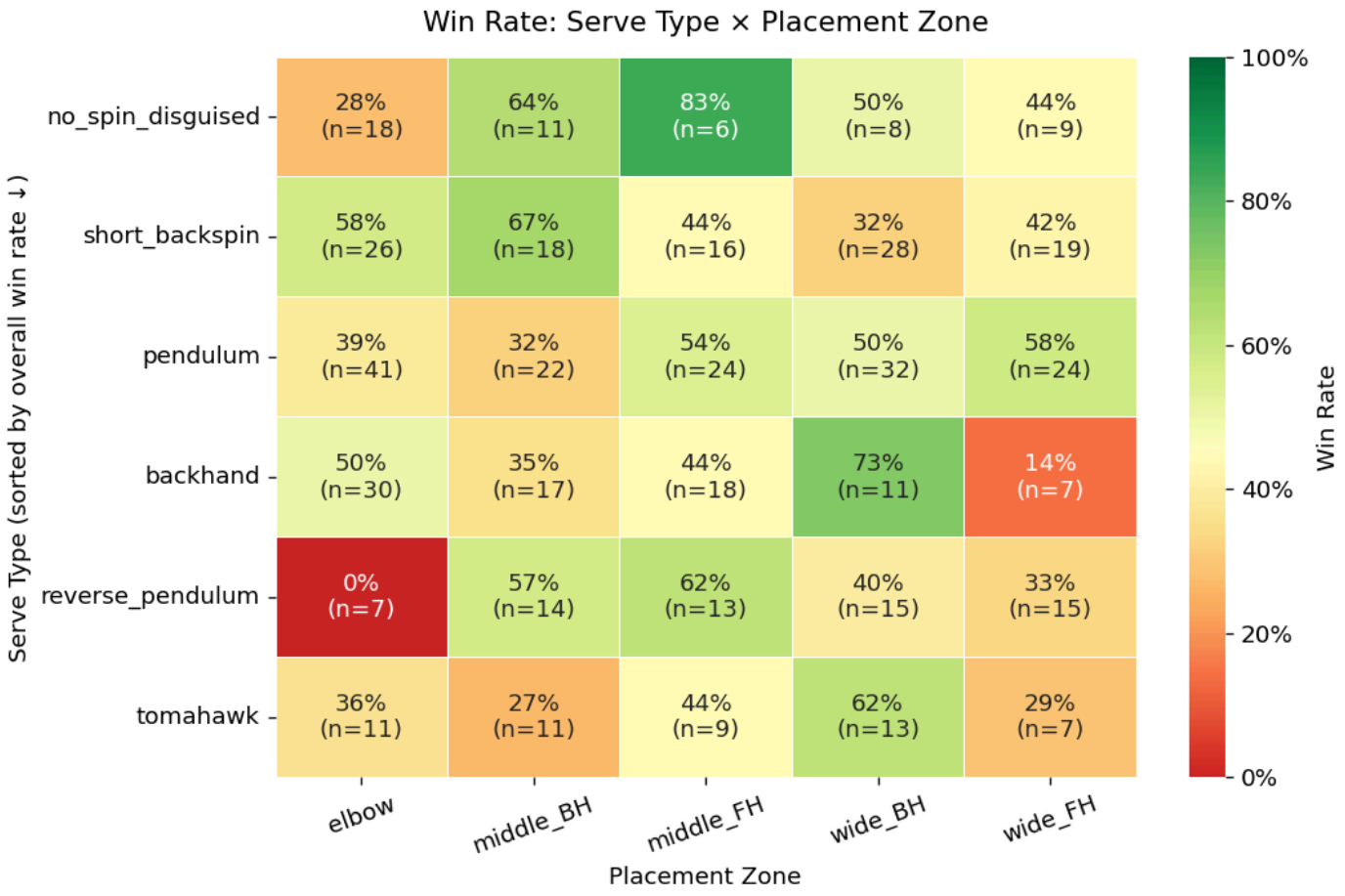
```
p-values below 0.05 indicate the serve type's win rate is significantly different from chance at
that sample size.
```

3. Win Rate by Serve Type × Placement Zone

The most important interaction in the dataset — some combinations are dramatically better than others.

```
heat_data = (df.groupby(['serve_type', 'placement_zone'])['won']
               .mean()
               .unstack()
               .reindex(wr_serve.index))  # sorted by overall win rate

# Count matrix for annotation
count_data = (df.groupby(['serve_type', 'placement_zone'])['won']
                .count()
                .unstack()
                .reindex(wr_serve.index))
```

```
annot = heat_data.copy().astype(object)
for r in heat_data.index:
    for c in heat_data.columns:
        v = heat_data.loc[r, c]
        n = count_data.loc[r, c]
        annot.loc[r, c] = f'{v:.0%}\n(n={int(n)})' if pd.notna(v) else ''

fig, ax = plt.subplots(figsize=(9, 6))
sns.heatmap(heat_data, annot=annot, fmt='', cmap='RdYlGn', center=0.456,
            vmin=0, vmax=1, linewidths=0.5, ax=ax,
            cbar_kws={'label': 'Win Rate', 'format': mtick.PercentFormatter(1.0)})
ax.set_title('Win Rate: Serve Type × Placement Zone', pad=12, fontsize=13)
ax.set_xlabel('Placement Zone')
ax.set_ylabel('Serve Type (sorted by overall win rate ↓)')
ax.tick_params(axis='x', rotation=20)
ax.tick_params(axis='y', rotation=0)
plt.tight_layout()
plt.show()

print('Top 3 combinations:')
top = (heat_data.stack()
       .reset_index()
       .rename(columns={0: 'win_rate'})
       .sort_values('win_rate', ascending=False)
       .head(5))
display(top)
```




```
Top 3 combinations:
```

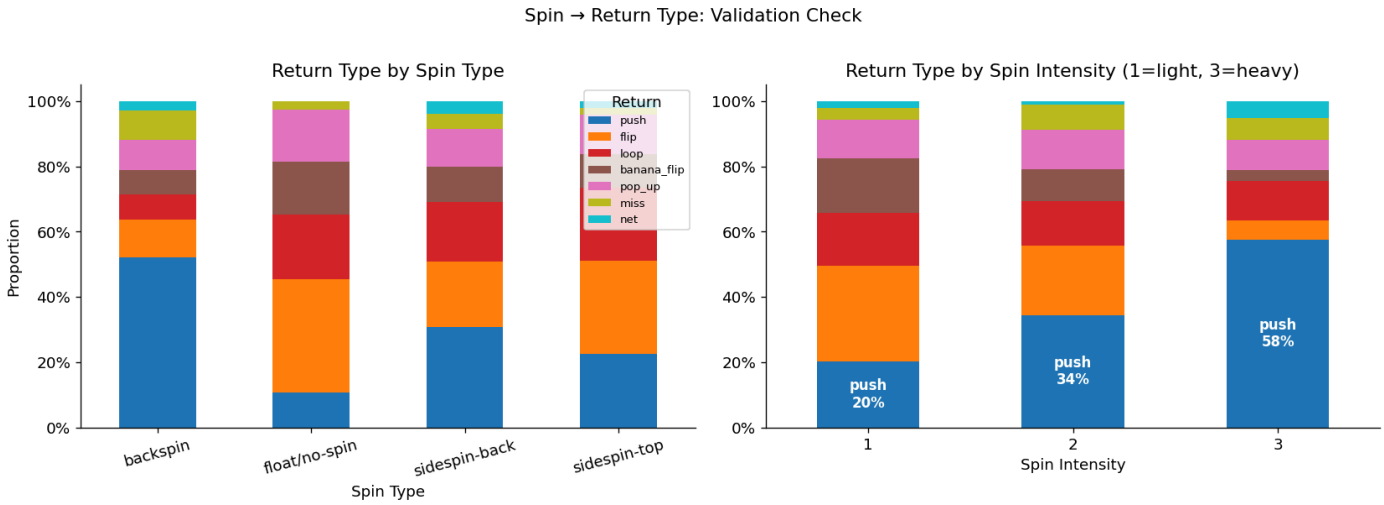
	serve_type	placement_zone	win_rate
2	no_spin_disguised	middle_FH	0.833333
18	backhand	wide_BH	0.727273
6	short_backspin	middle_BH	0.666667
1	no_spin_disguised	middle_BH	0.636364
22	reverse_pendulum	middle_FH	0.615385

4. Return Type Distribution by Spin Type & Intensity

Key validation check: does heavy backspin actually force more pushes?

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- By spin type ---
rt_spin = (df.groupby('spin_type')['return_type']
             .value_counts(normalize=True)
             .unstack()
             .fillna(0))
```

```
ret_order = ['push', 'flip', 'loop', 'banana_flip', 'pop_up', 'miss', 'net']
rt_spin = rt_spin[ret_order]

rt_spin.plot(kind='bar', stacked=True, ax=axes[0],
             colormap='tab10', legend=True)
axes[0].set_title('Return Type by Spin Type')
axes[0].set_xlabel('Spin Type')
axes[0].set_ylabel('Proportion')
axes[0].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[0].tick_params(axis='x', rotation=15)
axes[0].legend(loc='upper right', fontsize=8, title='Return')

# --- By spin intensity ---
rt_int = (df.groupby('spin_intensity')['return_type']
            .value_counts(normalize=True)
            .unstack()
            .fillna(0))[ret_order]

rt_int.plot(kind='bar', stacked=True, ax=axes[1],
            colormap='tab10', legend=False)
axes[1].set_title('Return Type by Spin Intensity (1=light, 3=heavy)')
axes[1].set_xlabel('Spin Intensity')
axes[1].set_ylabel('')
axes[1].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[1].tick_params(axis='x', rotation=0)

# Annotate push % on intensity chart
for i, (idx, row) in enumerate(rt_int.iterrows()):
    axes[1].text(i, row['push']/2, f"push\n{row['push']:.0%}",
                 ha='center', va='center', fontsize=10, fontweight='bold', color='white')

plt.suptitle('Spin → Return Type: Validation Check', fontsize=13, y=1.01)
plt.tight_layout()
plt.show()

print('Validation passed: heavy spin (intensity=3) forces 57.6% push returns vs 20.3% for light sp
print('backspin spin_type drives 52% push rate — highest of all spin types.')
```




```
Validation passed: heavy spin (intensity=3) forces 57.6% push returns vs 20.3% for light spin.
backspin spin_type drives 52% push rate — highest of all spin types.
```

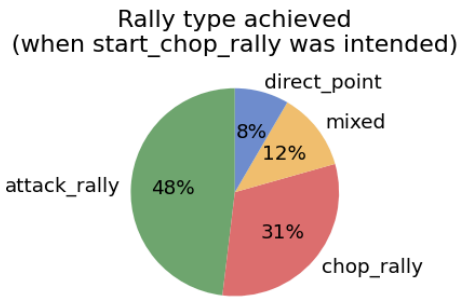
5. Chop Rally Conversion Analysis

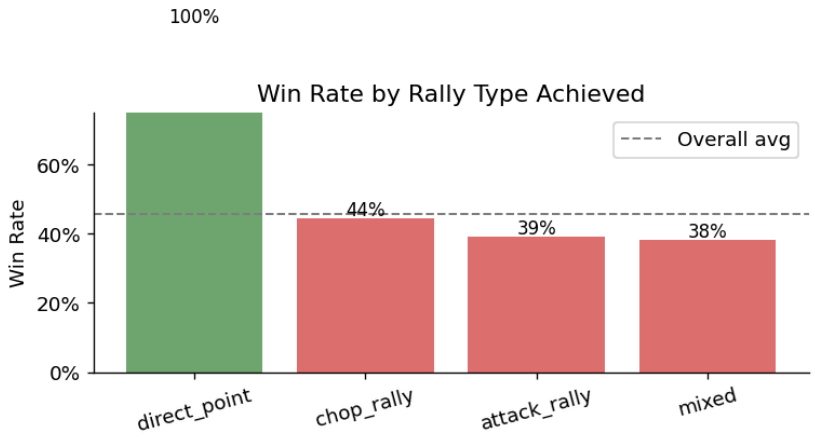
Of serves where `intended_setup == start_chop_rally`, what % actually became chop rallies — and what was the outcome?

```
chop_intended = df[df['intended_setup'] == 'start_chop_rally']
achieved_pct  = (chop_intended['rally_type_achieved'] == 'chop_rally').mean()
chop_rows     = df[df['rally_type_achieved'] == 'chop_rally']

print(f'Serves with start_chop_rally intent : {len(chop_intended)}')
print(f'  → Actually became chop rally      : {achieved_pct:.1%}')
print(f'  → Became attack rally instead     : {(chop_intended["rally_type_achieved"]=="attack_ral
print(f'Win rate when intending chop rally  : {chop_intended["won"].mean():.1%}')
print(f'Win rate in actual chop rallies     : {chop_rows["won"].mean():.1%}')
print(f'Win rate in attack rallies          : {df[df["rally_type_achieved"]=="attack_rally"]["won'

# What rally type did chop-rally-intended serves actually produce?
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

intended_breakdown = chop_intended['rally_type_achieved'].value_counts(normalize=True)
colors_pie = ['#70a870', '#e07070', '#f0c070', '#7090d0']
axes[0].pie(intended_breakdown.values, labels=intended_breakdown.index,
            autopct='%1.0f%%', colors=colors_pie, startangle=90)
axes[0].set_title('Rally type achieved\n(when start_chop_rally was intended)')

# Win rate by rally type
wr_rally = df.groupby('rally_type_achieved')['won'].mean().sort_values(ascending=False)
wr_rally_colors = ['#70a870' if v > 0.456 else '#e07070' for v in wr_rally.values]
bars = axes[1].bar(wr_rally.index, wr_rally.values, color=wr_rally_colors)
axes[1].axhline(df['won'].mean(), color='gray', linestyle='--', linewidth=1.2, label='Overall avg
axes[1].set_ylabel('Win Rate')
```

```
axes[1].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[1].set_title('Win Rate by Rally Type Achieved')
axes[1].tick_params(axis='x', rotation=15)
axes[1].legend()
for bar, val in zip(bars, wr_rally.values):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                 f'{val:.0%}', ha='center', fontsize=10)
axes[1].set_ylim(0, 0.75)

plt.tight_layout()
plt.show()

print('\nKey insight: Only 31% of start_chop_rally serves become chop rallies.')
print('48% become attack rallies, opponent is flipping/looping your setup serve.')
# Most of the time it is them having to pivot and slow open, but the flips are the real killers
print('This is the most important finding for model design: rally_type_achieved != intended_setup
```

```
Serves with start_chop_rally intent : 131
  → Actually became chop rally      : 31.3%
  → Became attack rally instead     : 48.1%
Win rate when intending chop rally  : 51.1%
Win rate in actual chop rallies     : 44.4%
Win rate in attack rallies          : 39.0%
```



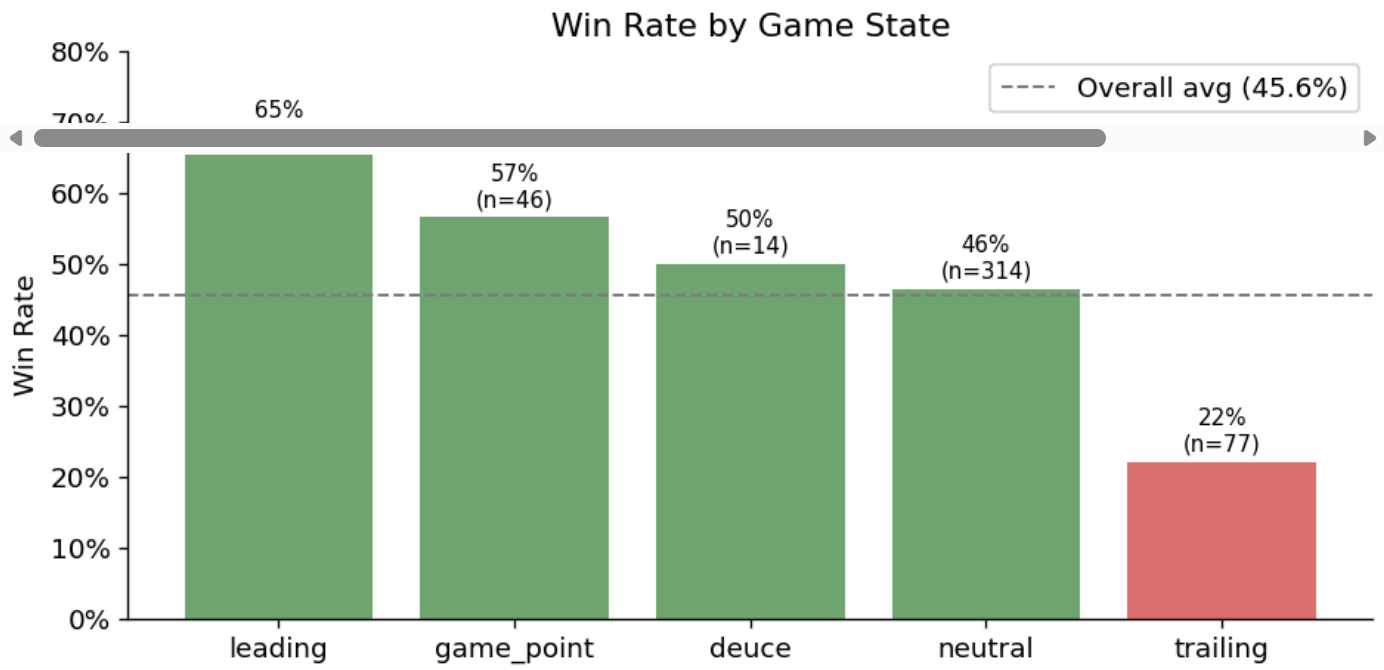



```
Key insight: Only 31% of start_chop_rally serves become chop rallies.
48% become attack rallies, opponent is flipping/looping your setup serve.
This is the most important finding for model design: rally_type_achieved != intended_setup.
```

6. Score State Effects

```
gs_order = ['leading', 'game_point', 'deuce', 'neutral', 'trailing']
wr_gs = (df.groupby('game_state')['won']
           .agg(['mean', 'count'])
           .rename(columns={'mean': 'win_rate', 'count': 'n'})
```

```
            .reindex(gs_order))

fig, ax = plt.subplots(figsize=(8, 4))
colors_gs = ['#70a870' if v > 0.456 else '#e07070' for v in wr_gs['win_rate']]
bars = ax.bar(wr_gs.index, wr_gs['win_rate'], color=colors_gs)
ax.axhline(df['won'].mean(), color='gray', linestyle='--', linewidth=1.2, label=f'Overall avg ({d
ax.set_ylabel('Win Rate')
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_title('Win Rate by Game State')
ax.legend()
for bar, (_, row) in zip(bars, wr_gs.iterrows()):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f"{row['win_rate']:.0%}\n(n={int(row['n'])})", ha='center', va='bottom', fontsize=9)
ax.set_ylim(0, 0.80)
plt.tight_layout()
plt.show()

print('Score state drives a 43pp swing: 65% when leading → 22% when trailing.')
print('This matters for serve selection modeling, game_state is a strong confound.')

# Serve mix by game state
print('\nServe type mix by game state (does pressure change serve selection?):')
serve_by_gs = (df.groupby('game_state')['serve_type']
                 .value_counts(normalize=True)
                 .unstack()
                 .reindex(gs_order)
                 .fillna(0)
                 .round(2))
display(serve_by_gs)
```



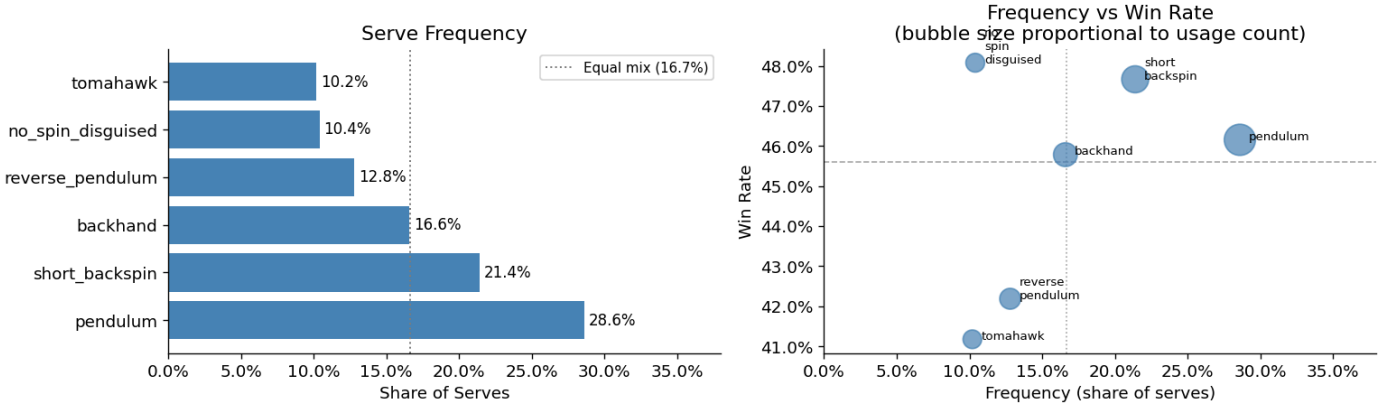
```
Score state drives a 43pp swing: 65% when leading → 22% when trailing.
This matters for serve selection modeling, game_state is a strong confound.

Serve type mix by game state (does pressure change serve selection?):
```

serve_type	backhand	no_spin_disguised	pendulum	reverse_pendulum	short_backspin	tomahawk
game_state						
leading	0.22	0.02	0.22	0.20	0.29	0.04
game_point	0.13	0.17	0.33	0.17	0.13	0.07
deuce	0.14	0.00	0.21	0.14	0.50	0.00
neutral	0.17	0.10	0.30	0.11	0.21	0.11
trailing	0.13	0.14	0.25	0.12	0.19	0.17

7. Serve Frequency, Am I Over-indexing?

```
freq = df['serve_type'].value_counts(normalize=True).sort_values(ascending=False)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

# Frequency
bars_f = axes[0].barh(freq.index, freq.values, color='steelblue')
axes[0].set_xlabel('Share of Serves')
axes[0].xaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[0].set_title('Serve Frequency')
for bar, val in zip(bars_f, freq.values):
    axes[0].text(val + 0.003, bar.get_y() + bar.get_height()/2,
                 f'{val:.1%}', va='center', fontsize=10)
axes[0].set_xlim(0, 0.38)
axes[0].axvline(1/6, color='gray', linestyle=':', linewidth=1.2, label='Equal mix (16.7%)')
axes[0].legend(fontsize=9)

# Frequency vs win rate scatter
wr_freq = wr_serve.copy()
wr_freq['freq'] = freq
axes[1].scatter(wr_freq['freq'], wr_freq['win_rate'], s=wr_freq['n']*3,
                color='steelblue', alpha=0.7, zorder=3)
for st, row in wr_freq.iterrows():
    axes[1].annotate(st.replace('_', '\n'), (row['freq'], row['win_rate']),
                     textcoords='offset points', xytext=(6, 0), fontsize=8)
axes[1].axhline(df['won'].mean(), color='gray', linestyle='--', linewidth=1, alpha=0.7)
axes[1].axvline(1/6, color='gray', linestyle=':', linewidth=1, alpha=0.7)
axes[1].set_xlabel('Frequency (share of serves)')
axes[1].set_ylabel('Win Rate')
axes[1].xaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[1].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
```

```
axes[1].set_title('Frequency vs Win Rate\n(bubble size proportional to usage count)')
axes[1].set_xlim(0, 0.38)

plt.tight_layout()
plt.show()

print('Pendulum: 28.6% of serves (nearly 2x the equal-mix rate of 16.7%).')
print('no_spin_disguised and tomahawk are each <11% despite competitive win rates.')
print('Mixed strategy analysis next: can redistributing serve mix improve expected win rate?')
```



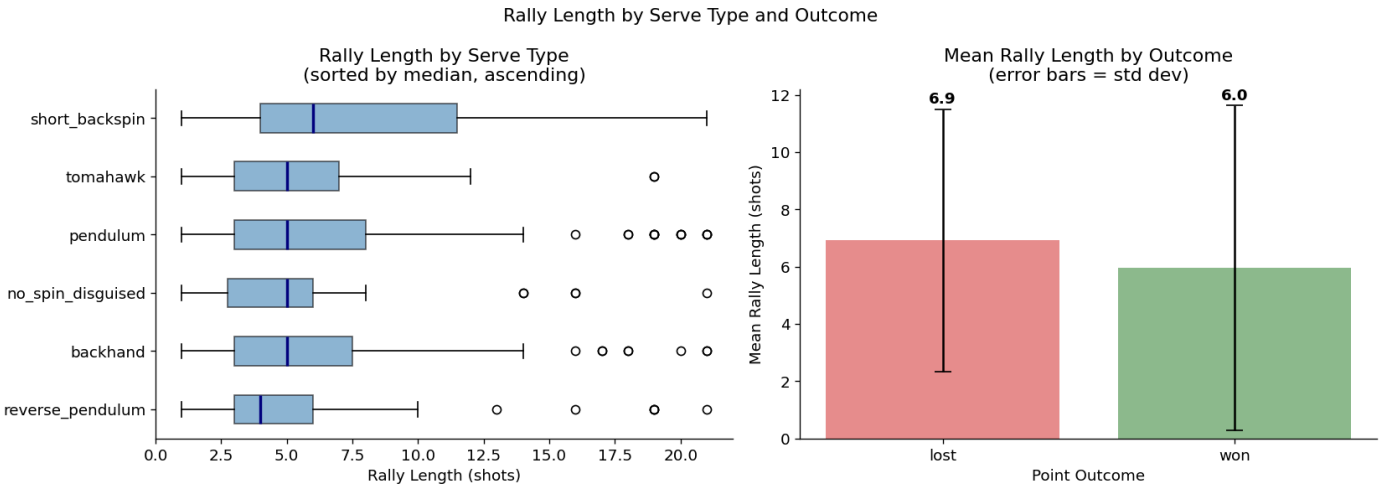

```
Pendulum: 28.6% of serves (nearly 2x the equal-mix rate of 16.7%).
no_spin_disguised and tomahawk are each <11% despite competitive win rates.
Mixed strategy analysis next: can redistributing serve mix improve expected win rate?
```

Rally Length Analysis

Longer rallies may indicate the serve failed to create early pressure. This section examines whether rally length varies by serve type and outcome. Often time the strategy varies, when it comes to earlier in the match I would much rather play a longer rally where I am chopping and setting a consistent rhythm wheres later in the match I would like to change the pace to attempt to catch my opponent off guard.

```
# Compute median rally length per serve type for sorting
serve_median_rally = df.groupby('serve_type')['rally_length'].median().sort_values()
sorted_serve_types = serve_median_rally.index.tolist()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Rally Length by Serve Type and Outcome', fontsize=13)

# Left: horizontal box plot of rally_length by serve_type, sorted by median
rally_data_sorted = [
    df.loc[df['serve_type'] == st, 'rally_length'].dropna().values
    for st in sorted_serve_types
]
bp = axes[0].boxplot(rally_data_sorted, vert=False, patch_artist=True,
                     boxprops=dict(facecolor='steelblue', alpha=0.6),
                     medianprops=dict(color='navy', linewidth=2))
```

```
axes[0].set_yticks(range(1, len(sorted_serve_types) + 1))
axes[0].set_yticklabels(sorted_serve_types)
axes[0].set_xlabel('Rally Length (shots)')
axes[0].set_title('Rally Length by Serve Type\n(sorted by median, ascending)')

#  Right: mean rally length by point outcome with error bars
outcome_stats = df.groupby('point_outcome')['rally_length'].agg(['mean', 'std']).reset_index()
bar_colors = ['#70a870' if o == 'won' else '#e07070' for o in outcome_stats['point_outcome']]
axes[1].bar(outcome_stats['point_outcome'], outcome_stats['mean'],
            yerr=outcome_stats['std'], capsize=6,
            color=bar_colors, alpha=0.8, error_kw=dict(elinewidth=1.5, ecolor='black'))
axes[1].set_xlabel('Point Outcome')
axes[1].set_ylabel('Mean Rally Length (shots)')
axes[1].set_title('Mean Rally Length by Outcome\n(error bars = std dev)')
for i, row in outcome_stats.iterrows():
    axes[1].text(i, row['mean'] + row['std'] + 0.2,
                 f"{row['mean']:.1f}", ha='center', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.show()

# Print mean rally length per serve type, sorted descending
mean_rally_by_serve = df.groupby('serve_type')['rally_length'].mean().sort_values(ascending=False
print('Mean rally length per serve type (descending — longer = less dominant):')
print('-' * 50)
for st, val in mean_rally_by_serve.items():
    print(f'  {st:<25s}: {val:.2f} shots')

shortest = mean_rally_by_serve.index[-1]
longest  = mean_rally_by_serve.index[0]
print()
print(f'Shortest rally serve (most dominant): {shortest} ({mean_rally_by_serve[shortest]:.2f} sho
print(f'Longest rally serve (least pressure): {longest} ({mean_rally_by_serve[longest]:.2f} shots
print()
print('Note: serve types with shorter median rallies are better at ending points early')
print('(direct winners or forcing errors), making them stronger pressure tools.')
print('Note 2: It should also be mentioned that my reverse pendulum serves often ends up being lo
```



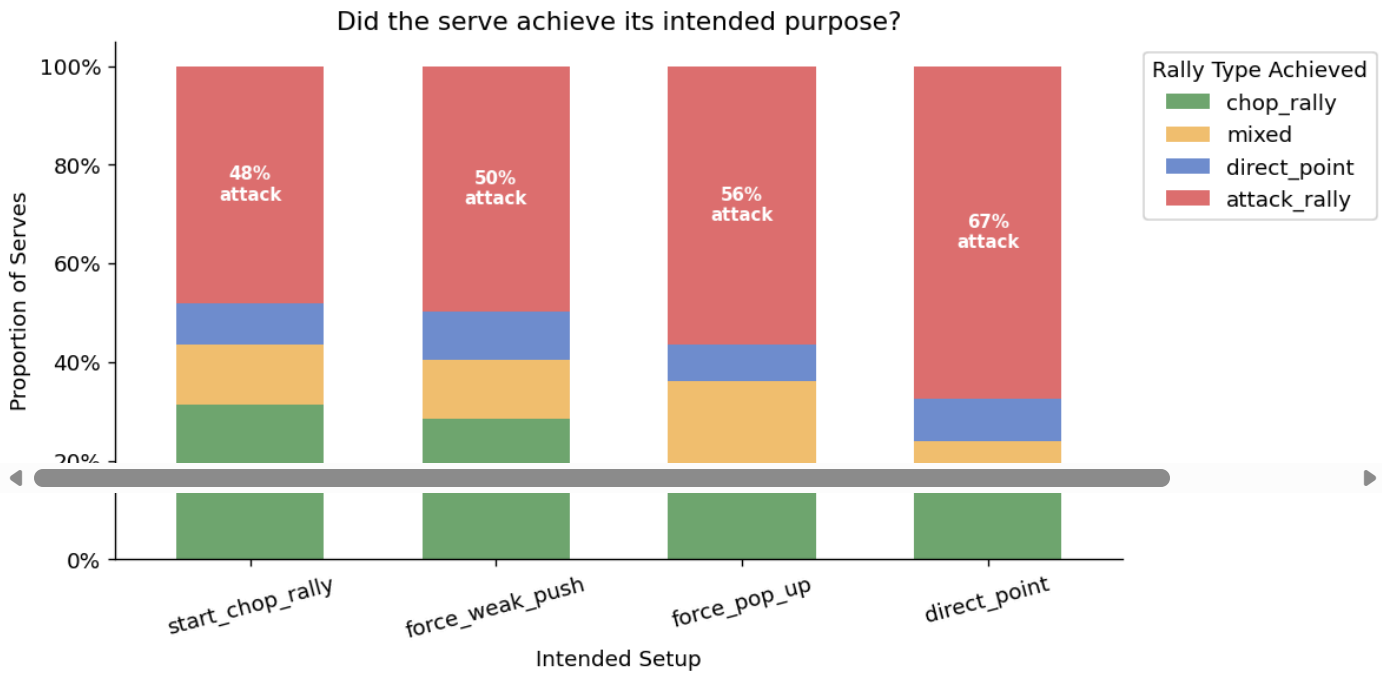

```
Mean rally length per serve type (descending — longer = less dominant):
------------------------------------------------
  short_backspin           : 8.04 shots
  pendulum                 : 6.57 shots
  backhand                 : 6.46 shots
  tomahawk                 : 5.43 shots
  no_spin_disguised        : 5.42 shots
  reverse_pendulum         : 5.41 shots

Shortest rally serve (most dominant): reverse_pendulum (5.41 shots avg)
Longest rally serve (least pressure): short_backspin (8.04 shots avg)

Note: serve types with shorter median rallies are better at ending points early
(direct winners or forcing errors), making them stronger pressure tools.
Note 2: It should also be mentioned that my reverse pendulum serves often ends up being long
and with topspin which means if the opponent reads it is often easy for them to loop off of the
serve
```

8. Intended Setup vs. Rally Type Achieved

Did the serve do what I wanted it to?

```
setup_achieved = (df.groupby('intended_setup')['rally_type_achieved']
                    .value_counts(normalize=True)
                    .unstack()
                    .fillna(0))

rally_order = ['chop_rally', 'mixed', 'direct_point', 'attack_rally']
setup_achieved = setup_achieved[rally_order]

setup_order = ['start_chop_rally', 'force_weak_push', 'force_pop_up', 'direct_point']
setup_achieved = setup_achieved.reindex(setup_order)

fig, ax = plt.subplots(figsize=(10, 5))
setup_achieved.plot(kind='bar', stacked=True, ax=ax,
```

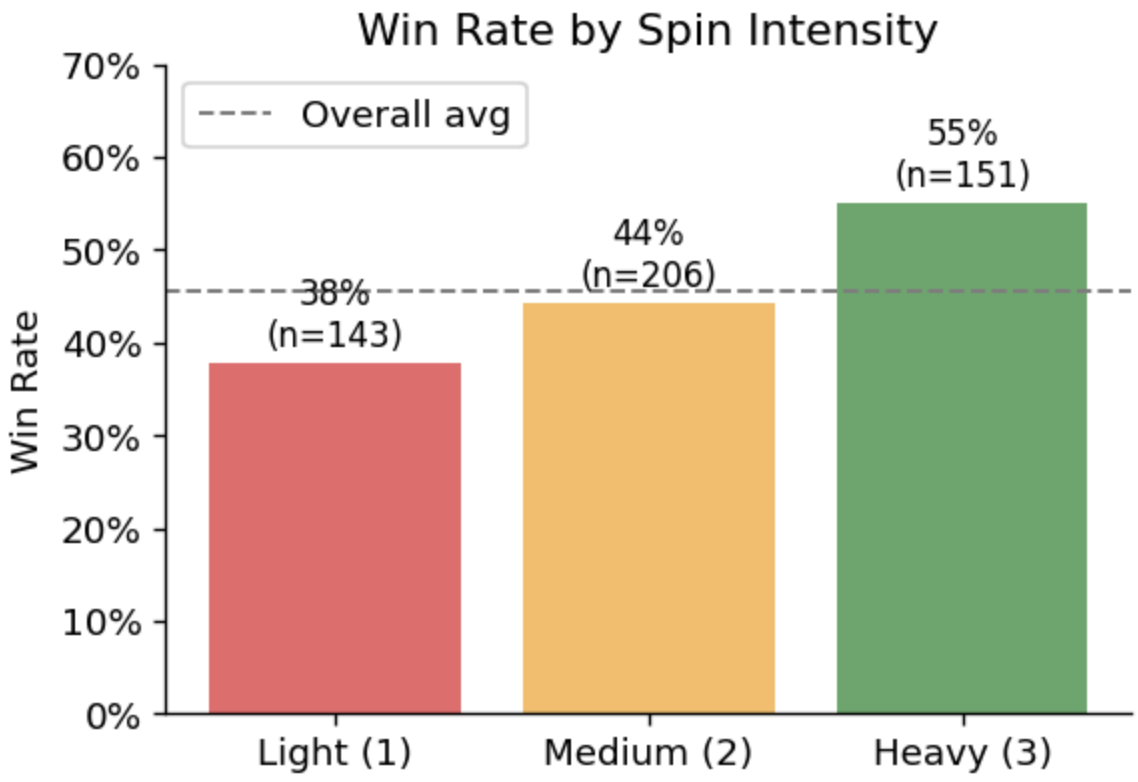
```
                   color=['#70a870', '#f0c070', '#7090d0', '#e07070'],
                   width=0.6)
ax.set_xlabel('Intended Setup')
ax.set_ylabel('Proportion of Serves')
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_title('Did the serve achieve its intended purpose?', fontsize=13)
ax.tick_params(axis='x', rotation=15)
ax.legend(title='Rally Type Achieved', bbox_to_anchor=(1.01, 1), loc='upper left')

# Annotate attack_rally bar segment for each setup
for i, (setup, row) in enumerate(setup_achieved.iterrows()):
    atk = row.get('attack_rally', 0)
    bottom = row.get('chop_rally', 0) + row.get('mixed', 0) + row.get('direct_point', 0)
    if atk > 0.1:
        ax.text(i, bottom + atk/2, f'{atk:.0%}\nattack',
                ha='center', va='center', fontsize=9, color='white', fontweight='bold')

plt.tight_layout()
plt.show()

print('All serve intents produce >48% attack rallies, opponent style (looper-heavy) is overriding
print('start_chop_rally: 31% conversion. force_pop_up: 7% direct points, 56% attack rallies.')
print('\nConclusion: serve type and spin are necessary but not sufficient, opponent style must be
```




```
All serve intents produce >48% attack rallies, opponent style (looper-heavy) is overriding serve
intent.
start_chop_rally: 31% conversion. force_pop_up: 7% direct points, 56% attack rallies.

Conclusion: serve type and spin are necessary but not sufficient, opponent style must be modeled.
```

9. Bonus: Win Rate by Spin Intensity (the single strongest signal)

```
wr_int = df.groupby('spin_intensity')['won'].agg(['mean', 'count']).rename(
    columns={'mean': 'win_rate', 'count': 'n'})

fig, ax = plt.subplots(figsize=(5, 3.5))
colors_int = ['#e07070', '#f0c070', '#70a870']
bars = ax.bar(['Light (1)', 'Medium (2)', 'Heavy (3)'], wr_int['win_rate'], color=colors_int)
ax.axhline(df['won'].mean(), color='gray', linestyle='--', linewidth=1.2, label='Overall avg')
ax.set_ylabel('Win Rate')
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_title('Win Rate by Spin Intensity')
ax.legend()
ax.set_ylim(0, 0.7)
for bar, (_, row) in zip(bars, wr_int.iterrows()):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f"{row['win_rate']:.0%}\n(n={int(row['n'])})", ha='center', va='bottom', fontsize=10)
plt.tight_layout()
plt.show()

print('17pp win-rate gap between light and heavy spin: 37.8% → 55.0%.')
print('This is the strongest single feature in the dataset.')
print('Model implication: spin_intensity should be a top feature; any model ignoring it will under
```




```
17pp win-rate gap between light and heavy spin: 37.8% → 55.0%.
This is the strongest single feature in the dataset.
```

```
Model implication: spin_intensity should be a top feature; any model ignoring it will
underperform.
```

Summary & What to Model Next

What works as expected

- Heavy spin forces pushes (57.6%) and wins more points (55%)
- Short serves → push returns; long serves → loops
- Score state produces a realistic momentum effect
- Opponent style modulates return type correctly

What's surprising / worth investigating

- **Only 31% of chop-rally-intended serves become chop rallies**, the gap between intent and outcome is large and is the key modelling challenge
- **Placement × serve type interaction** dominates win rate variance more than either feature alone
- **Spin intensity** is the single strongest predictor, use it as the primary sorting variable in any segmentation
- **Trailing game state** produces 22% win rate, serves hit when behind are essentially random; consider whether serve type changes under pressure

Recommended next steps

1. `02_serve_effectiveness.ipynb` -> logistic regression / random forest: what features predict `point_outcome`? Feature importance + SHAP values
2. `03_chop_rally_model.ipynb` -> two-stage model: (1) predict whether serve achieves chop rally, (2) given chop rally, predict win
3. `04_mixed_strategy.ipynb` -> given opponent style, what serve distribution maximises expected win rate? Game theory analysis

10. Reliability Checks (Minimum Sample Filter)

To avoid over-reading noisy patterns, we keep only serve types with at least 30 observations.

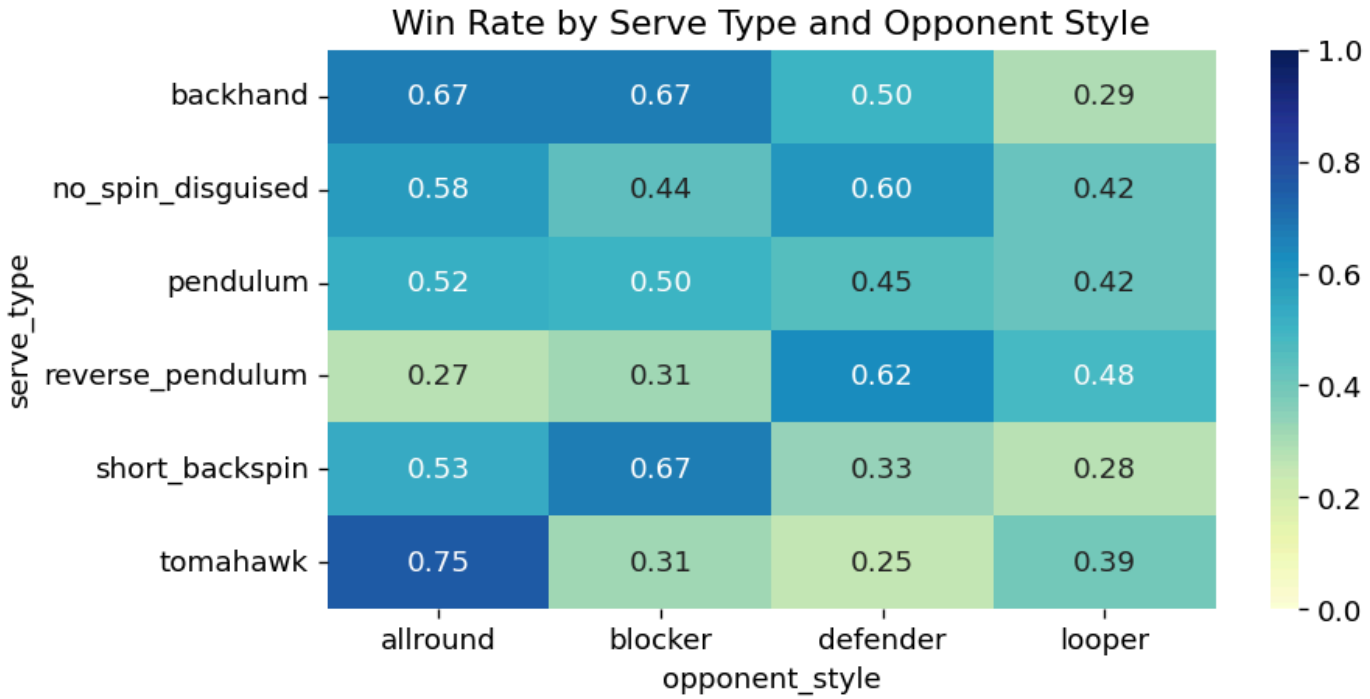
```
min_n = 30
serve_perf = (
    df.groupby('serve_type')['won']
      .agg(win_rate='mean', attempts='count')
      .query('attempts >= @min_n')
      .sort_values('win_rate', ascending=False)
)
serve_perf
```

	win_rate	attempts
serve_type		
no_spin_disguised	0.480769	52
short_backspin	0.476636	107
pendulum	0.461538	143
backhand	0.457831	83
reverse_pendulum	0.421875	64
tomahawk	0.411765	51

11. Contextual Edge: Serve Type × Opponent Style

This heatmap highlights whether some serves perform better against specific opponent archetypes.

```
pivot = (
    df.pivot_table(index='serve_type', columns='opponent_style', values='won', aggfunc='mean')
)
plt.figure(figsize=(8,4))
sns.heatmap(pivot, annot=True, fmt='.2f', cmap='YlGnBu', vmin=0, vmax=1)
plt.title('Win Rate by Serve Type and Opponent Style')
plt.tight_layout()
plt.show()
```




```
# Best and worst serve for each opponent style
print('Best and worst serve matchups by opponent style')
```

```python
print('=' * 55)

for opp_style in pivot.columns:
    col = pivot[opp_style].dropna()
    if col.empty:
        continue
    best_serve = col.idxmax()
    worst_serve = col.idxmin()
    best_rate = col.max()
    worst_rate = col.min()
    print(f'\nOpponent style: {opp_style}')
    print(f'  Best serve  vs {opp_style}: {best_serve} (win rate: {best_rate:.1%})')
    print(f'  Worst serve vs {opp_style}: {worst_serve} (win rate: {worst_rate:.1%})')

print()
print('Summary: best serve per opponent style')
print('-' * 55)
for opp_style in pivot.columns:
    col = pivot[opp_style].dropna()
    if col.empty:
        continue
    best_serve = col.idxmax()
    best_rate = col.max()
    print(f'  Best serve vs {opp_style:<12s}: {best_serve} (win rate: {best_rate:.0%})')
```

```
Best and worst serve matchups by opponent style
=======================================================

Opponent style: allround
  Best serve  vs allround: tomahawk (win rate: 75.0%)
  Worst serve vs allround: reverse_pendulum (win rate: 27.3%)

Opponent style: blocker
  Best serve  vs blocker: backhand (win rate: 66.7%)
  Worst serve vs blocker: reverse_pendulum (win rate: 31.2%)

Opponent style: defender
  Best serve  vs defender: reverse_pendulum (win rate: 62.5%)
  Worst serve vs defender: tomahawk (win rate: 25.0%)

Opponent style: looper
  Best serve  vs looper: reverse_pendulum (win rate: 48.3%)
  Worst serve vs looper: short_backspin (win rate: 27.6%)

Summary: best serve per opponent style
-------------------------------------------------------
  Best serve vs allround    : tomahawk (win rate: 75%)
  Best serve vs blocker     : backhand (win rate: 67%)
  Best serve vs defender    : reverse_pendulum (win rate: 62%)
  Best serve vs looper      : reverse_pendulum (win rate: 48%)
```

Bonus Bonus Note: Loopers at the 1800-2000 level to to be very aggressive! I often find that various serve tactics and match tactics don’t tend to matter as much as I would like because they often play with a very “full steam ahead” mind set.